

Predict Once, Query Many: World Models that Forecast Conditional Trajectory Abstractions

Anonymous CVPR submission

Paper ID *****

Abstract

001 *World models for visual control usually roll the future out*
002 *one time step at a time, predicting an observation or its*
003 *latent representation for every future step, and leave it to a*
004 *planner to extract what matters for a decision. Prediction*
005 *cost grows with the horizon and the number of tokens per*
006 *step, and most of each predicted state restates what the*
007 *observed history already shows. We propose Conditional*
008 *Trajectory Abstraction (CTA), a joint-embedding predictive*
009 *world model whose prediction target is not one latent state*
010 *per time step but a short discrete code of an action chunk's*
011 *entire future segment, conditioned on the observed history.*
012 *During training only, a target encoder compresses the actual*
013 *future segment into a few tokens given the history. Two*
014 *heads see the history and the code but never the action: a*
015 *reader that ranks candidate chunks by progress toward a*
016 *goal image, and a decoder that reconstructs the future's*
017 *frozen image features, which keeps the code a description*
018 *of the future rather than a score. The world model predicts*
019 *the code from the history and a candidate chunk against*
020 *a stop-gradient target, and an optional predictability term*
021 *co-designs the code with the world model. At deployment, a*
022 *frozen policy proposes candidate chunks, the world model*
023 *predicts one code per candidate, and the reader scores each*
024 *code against any number of goal images without predicting*
025 *again. Task-progress labels enter only the reader's training*
026 *loss; the world model never sees a label, and nothing uses*
027 *labels at test time. On PushT and LIBERO-Goal, CTA [main*
028 *result placeholder].*

029 1. Introduction

030 A world model lets an agent evaluate an action before taking
031 it. The dominant recipe for visual control rolls the future out
032 one time step at a time, predicting pixels, discrete tokens, or
033 the latent features of a pretrained encoder for every future
034 step, and a planner then scores the imagined states against
035 a goal [3, 17, 28, 39]. Joint-embedding predictive architec-

tures (JEPA) [2, 24] moved this recipe away from pixels. 036
DINO-WM [39] and V-JEPA 2-AC [3] predict, entirely in 037
latent space, the patch features of a frozen self-supervised 038
encoder for every future step, and plan by minimizing the 039
feature distance between a predicted state and a goal image. 040
Their prediction target, however, is still one latent state per 041
time step. The cost of evaluating one candidate grows with 042
the horizon and with the number of patch tokens per step, 043
and errors compound over the rollout. Most of every pre- 044
dicted state restates what the observed history already shows: 045
the scene layout, the appearance of objects, and everything 046
the action does not touch. 047

Predicting per-step states also leaves the planner with the 048
task of turning them into a decision, and a generic feature 049
distance is a poor measure of task progress. On PushT, the 050
proposals of a released diffusion policy often contain a better 051
chunk than the one it executes. Choosing the best of eight 052
proposals with a privileged physical oracle raises closed-loop 053
success by 14 points on 400 episodes. On a separate set of 054
100 episodes, where the oracle gains 15 points, choosing by 055
DINOv2 feature distance to goal images gains only 3, even 056
though the distance is computed on the *true* future frames. A 057
reader trained to rank the true futures by task progress gains 058
8.5 points on the 400 episodes (Sec. 5.2). A per-step latent 059
world model that plans with feature distance would inherit 060
the first ceiling even if it predicted perfectly. CTA instead 061
learns the reader, and asks how much of its gain survives 062
when the future is first compressed and then predicted. 063

The opposite extreme skips prediction and learns a *direct* 064
scorer that maps the history, a candidate action, and a goal 065
to a score [30]. A direct scorer is cheap, but it entangles 066
two different things: what the action will do, which does 067
not depend on the goal, and how the outcome relates to the 068
goal, which does. It must learn the consequences of actions 069
separately for every goal, and it has no reason to generalize 070
to goals it was not trained on. 071

This paper asks whether a world model can sit between 072
these extremes: *instead of rolling out a latent state for ev-* 073
ery future time step, can a world model predict a compact 074
abstraction of an entire future segment that is still sufficient 075

for decision making? Two observations make this plausible. First, a planner never needs the future as such, only the answers to a few questions about it, such as how much closer to the goal the segment ends. Second, whatever the observed history already determines need not be predicted again. Learned video compression exploits the same fact through *conditional coding*, where both the encoder and the decoder see the temporal context and bits are spent only on what the context cannot predict [25]. We apply this principle to the *unexecuted* future of an action, inside a joint-embedding predictive world model. All world models in this paper, ours and the baselines, predict in latent space and never generate images; they differ in their unit of prediction, one latent state per time step or one code per segment.

We propose *Conditional Trajectory Abstraction* (CTA), summarized in Fig. 1. During training only, a target encoder compresses the actual future segment of an action chunk into M discrete tokens S , conditioned on the observed history. Two heads read the code together with the history. A reader ranks the candidates that branch from the same state by progress toward a goal image. A feature decoder reconstructs the frozen features of the future frames, which keeps the code a description of the future rather than a score for the training goals. Neither head receives the action, so everything the action changes must pass through S . The world model is an action-conditioned predictor of S , trained against the stop-gradient output of the target encoder as in other joint-embedding predictive architectures. Optionally, the code is also co-designed to be predictable from the action. In policy-guided planning, a frozen pretrained policy proposes a bank of candidate chunks, the world model predicts one code per candidate, and the reader scores each code against the goal images. Since codes do not depend on the goal, one prediction per candidate serves any number of goals: *predict once, query many*.

Each ingredient of CTA has precedents in world models, video compression, and policy steering, so a new name is not a contribution by itself. Our claims are therefore stated as comparisons at matched history, proposal bank, and data, against the two extremes above and against controls that remove one ingredient at a time. Our contributions are as follows.

- We formulate the prediction target of a world model as a *conditional* code of an action’s future segment, bounded in rate and required to be predictable from the action, with per-step latent world models and direct scorers as its two limiting cases.
- We introduce CTA, a joint-embedding predictive world model whose target is a learned, history-conditioned, segment-level discrete code. It is trained with a goal-conditioned ranking reader, a conditional feature decoder, and an autoregressive code predictor. Task-progress labels enter only the reader’s training loss, and the world model

never sees a label.

- We define an attribution ladder that scores the same candidates from the actual future, from its code, and from the predicted code. It separates ranking lost to compression from ranking lost to prediction.
- On PushT [8, 14] and LIBERO-Goal [26] with frozen released policies, [result placeholder: CTA vs. policy only, direct scorer, per-step latent world model, and GPC; held-out goals].

2. Related Work

Latent and joint-embedding world models. Latent world models learn dynamics in a compact state space and plan or learn behaviors in imagination [17–19]. Token-based models cast dynamics as autoregressive modeling of discrete image tokens [28]. Δ -IRIS encodes only the stochastic change between consecutive frames given the past [29], which makes it the closest per-step relative of conditional coding. Joint-embedding predictive architectures predict the representation of a target from the representation of its context instead of reconstructing the target [2, 5, 24]. In control, DINO-WM [39] and V-JEPA 2-AC [3] predict the patch features of a frozen encoder [31] one time step at a time, and they plan by minimizing the feature distance between a predicted latent state and a goal image. CompACT [21] compresses each observation into as few as eight tokens to accelerate such planning, and GCQ [32] compresses observation–action sequences with an action-conditioned codebook. In all of these models the prediction target is fixed in advance: the observation at each time step, or a per-step latent representation of it. CTA keeps the joint-embedding recipe, a predictor trained against the stop-gradient output of a target encoder, but learns the target itself. The target is one code per action segment, conditioned on history that the reader also sees, and shaped by what decisions need. Compact per-step codes are therefore our most important control.

Temporally extended prediction. Option models [36] and multi-step or trajectory-level dynamics models [10, 23] predict the outcome of a temporally extended action without stepping through every intermediate state, and jumpy world models compose such predictions across timescales [13]. Successor features and their random-feature variants predict cumulative features of a policy or an action sequence [6, 7], and TD-Flow learns generative models of discounted future-state occupancy [12]. These models summarize the future as a state at a given time or as an order-free aggregate, and they predict it from scratch. CTA codes the whole segment, including its intermediate frames, and spends its bits only on what the observed history does not already determine.

Policy steering with world models. A frozen generative policy can be improved at deployment by sampling several proposals and selecting one with an external scorer.

Deployment: predict once, query many

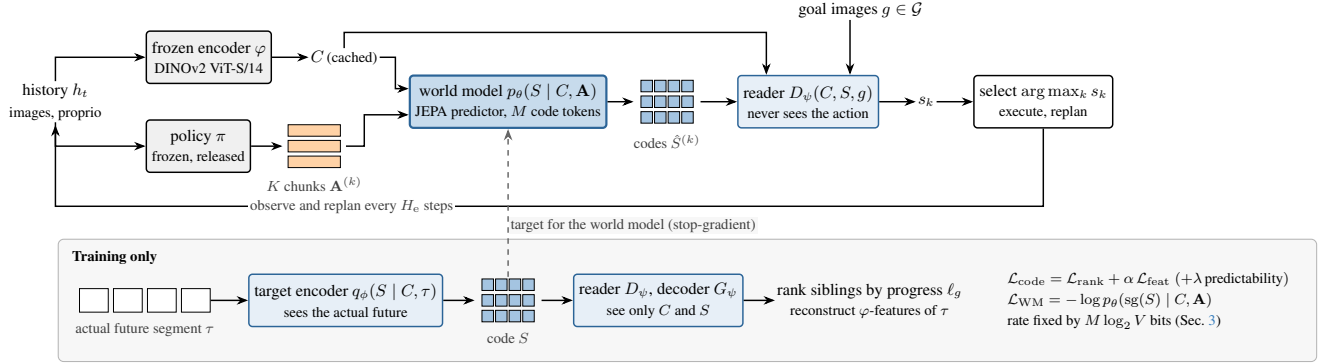


Figure 1. **Conditional Trajectory Abstraction.** *Top*: at each decision, the history is encoded once into a context C of frozen DINOv2 features, and a frozen policy proposes K action chunks. The world model, a joint-embedding predictor, forecasts for each chunk a short discrete code $\hat{S}$ of the chunk’s entire future segment, conditioned on C . A reader that sees C and the code, but never the action, scores each code against goal images. Since codes do not depend on the goal, one prediction per candidate serves any number of goals. *Bottom*: during training, a target encoder compresses the actual future segment into the code given C . A reader ranks sibling candidates by their progress labels, and a decoder reconstructs the future’s frozen features from (C, S) . The world model learns to predict the stop-gradient code; optionally, code and world model are co-designed for predictability.

GPC [33] ranks diffusion-policy proposals with a reward model applied to the future observations predicted by an action-conditioned world model; it is the closest system to ours. FOREWARN [37] aligns a vision–language model to latent predictions of candidate plans. DreamSteer [9] ranks imagined latent futures of VLA chunks with a language-conditioned value model. DynaGuide [11] steers denoising with a DINOv2-space dynamics model. Value- or verifier-based methods rank proposals without a world model [22, 30]. We adopt the same propose-then-select interface and, like GPC and V-GPS, train the scorer with task-progress labels. Our contribution concerns what the world model predicts. It predicts a goal-independent code that one reader evaluates for any goal image, where GPC predicts future observations that a reward model must read and a direct scorer outputs a score that must be recomputed for every goal.

Conditional coding and decision-aware compression.

Learned image and video codecs optimize a rate–distortion trade-off end to end [4]. DCVC [25] replaces residual coding with conditional coding, in which both encoder and decoder use the temporal context. Unlike the Wyner–Ziv setting [38], where side information is available only to the decoder, our encoder and heads both see the context. In reinforcement learning, value-equivalent models [16, 34] retain only what is needed to predict values, and value-equivalent sampling [1] formalizes decision-relevant lossy model compression. CTA differs in two ways. Its source is the unexecuted future of an action, which must be predicted before execution, so predictability is part of the design. Its distortion combines a decision term, the ranking of candidates toward a goal, with a generic term, the reconstruction of the future’s features, so

that the code is not specialized to the goals seen in training.

3. Problem Formulation

Setting. A robot observes images o_t and proprioceptive states q_t . A frozen visual encoder φ maps each image to a grid of patch features. At a decision time t , the context C_t consists of the features of the current and previous images and the recent proprioceptive states; it uses only present and past information. A candidate action chunk $\mathbf{A} = (a_t, \dots, a_{t+H_e-1})$ is executed for H_e steps. Its *future segment* τ consists of the frozen features of the images that executing $\mathbf{A}$ produces, $\varphi(o_{t+1}), \dots, \varphi(o_{t+H_e})$, subsampled in time, together with the final proprioceptive state. A world model must say something about τ *before* $\mathbf{A}$ is executed.

Policy-guided planning. At every decision, a frozen pre-trained policy π proposes K chunks $\mathbf{A}^{(1)}, \dots, \mathbf{A}^{(K)}$ from independent noise seeds, where $\mathbf{A}^{(1)}$ is the chunk the unmodified policy would execute. A scorer rates each proposal for the goal. The highest-scoring proposal is executed, with ties resolved in favor of $\mathbf{A}^{(1)}$, and the policy replans [9, 33]. Selecting among the proposals of a competent policy, rather than optimizing actions against the model, keeps candidates on the policy’s support. There, a learned model is less exposed to adversarial exploitation of its errors [15, 20]. Every compared model receives the same proposal bank. No scorer can recover an action that is absent from the bank, so we first measure how much a privileged oracle gains by selecting from it (Sec. 5.2).

Goals and labels. A goal is specified by a set of goal images $\mathcal{G}$, such as final frames of successful episodes of the task. For each candidate, the planner needs to know how

far its future segment advances the task toward $\mathcal{G}$. The K candidates branch from the same state, so only their *order* matters. During training, the simulator provides a progress label $\ell_g(\tau)$ for each training goal g . In PushT the label is the fraction of the target region the block covers. In LIBERO it is a dense progress score built from the goal’s BDDL predicates: object-to-target distance for placement predicates and normalized joint progress for articulation predicates. Labels train the reader only and are never available at test time; held-out goals are never labeled.

Conditional trajectory code. A *conditional trajectory code* is a sequence of M discrete tokens $S \in \{1, \dots, V\}^M$ produced by a target encoder $q_\phi(S | C, \tau)$, which sees the actual future and is used only during training. Two heads read the code together with the context, and neither receives $\mathbf{A}$. A reader $D_\psi(C, S, g)$ scores progress toward a goal image g , and a feature decoder $G_\psi(C, S)$ reconstructs τ . A world model $p_\theta(S | C, \mathbf{A})$ predicts the code from the context and the chunk. Since the heads never see the action, any information about its consequences must pass through S . The code solves a conditional compression problem,

$$\begin{aligned} \min_{q_\phi, \psi} \mathbb{E} [d_{\text{rank}}(D_\psi(C, S, g), \ell_g) + \alpha d_{\text{feat}}(G_\psi(C, S), \tau)] \\ \text{s.t. } I(\tau; S | C) \leq R, \end{aligned} \quad (1)$$

in which the context is available to both the encoder and the heads. We impose the rate through the code length. Because S is a deterministic function of (C, τ) , $I(\tau; S | C) = H(S | C) \leq M \log_2 V$, which is 128 bits per segment for $M=16$ and $V=256$. This count excludes the codebook, the heads, and the context features, which we report separately.

Predictability. At deployment the code is not observed but predicted, $\hat{S} \sim p_\theta(\cdot | C, \mathbf{A})$, and what matters is the ranking that $D_\psi(C, \hat{S}, g)$ induces. A code that solves (1) may spend bits on aspects of τ that the action does not determine, such as the outcome of an uncertain contact, and those bits cannot be forecast. The quantity that matters is the information the action carries about the code, $I(S; \mathbf{A} | C)$. We estimate it by the difference in cross-entropy between a context-only model $r_\omega(S | C)$ and the world model (Sec. 4.4). Optionally, the code is made more predictable during training (Sec. 4.5).

Two limiting cases. The formulation contains the two common designs as extremes. A *per-step latent world model*, such as DINO-WM, sets $S = \tau$. It applies no compression, predicts the patch features of all H_e future steps for every candidate, and needs a readout that is designed or learned on the predicted features. A *direct scorer* $D(C, \mathbf{A}, g)$ removes S . Prediction and reading are fused, so every candidate-goal pair needs a full evaluation, and generalization to new goals must be learned jointly with the effect of the action. CTA operates at a finite, explicit rate between them. We ask whether, at such a rate, predicted codes rank candidates as

	Goal-independent prediction	Per candidate and goal	Sequential steps
Per-step latent WM	$H_e \cdot P$ tokens	readout	H_e
Endpoint latent	P tokens	readout	1
Direct scorer	none	full model	1
CTA (ours)	M tokens	reader	M

Table 1. **Design properties** (not results) of models that evaluate a candidate chunk. All four are latent models; P is the number of patch tokens per step (256 here), and $M \in \{8, 16, 32\}$ is the number of code tokens. A direct scorer has no goal-independent part, so its whole cost recurs for every goal. CTA’s M autoregressive steps are short but not automatically cheaper than H_e per-step predictions, so we measure latency rather than assume it.

well as a per-step latent model at lower cost, and whether they transfer to held-out goals better than a direct scorer. Table 1 summarizes the design properties.

4. Conditional Trajectory Abstraction

CTA is a joint-embedding predictive world model. A target encoder turns the actual future into a code (Sec. 4.2), two heads read the code (Sec. 4.3), and a predictor forecasts the code from the action (Sec. 4.4). Section 4.5 describes training, Sec. 4.6 planning, and Sec. 4.7 implementation details.

4.1. Inputs

Images are embedded by a frozen DINOv2 ViT-S/14 [31], following DINO-WM [39]. We keep the full 16×16 patch grid and project the patch features to 128 dimensions with a PCA fitted on training frames. The context C contains the patch tokens of the current image, the tokens of the previous image pooled to an 8×8 grid, and the proprioceptive states at both times. The future segment τ contains the tokens of the segment’s final image on the full grid, the tokens of intermediate images pooled to 8×8 with temporal embeddings, and the final proprioceptive state. Features are computed once per image and shared by all candidates and goals. Each module embeds C and τ with its own linear projections and learned position and type embeddings.

4.2. Target encoder

The target encoder $q_\phi(S | C, \tau)$ exists only during training. M learned segment queries cross-attend jointly to the future tokens and the context tokens in a Transformer decoder. Each output is projected to three dimensions and quantized by finite scalar quantization (FSQ) [27] with levels (8, 8, 4), an implicit vocabulary of $V=256$ per token. Because the queries attend to the context, the encoder can learn which parts of the future the history already implies, instead of relying on a hand-designed residual such as the difference

between future and current features. The code describes the whole segment; it is not a set of M tokens per time step.

4.3. Reader and feature decoder

Reader. The reader $D_\psi(C, S, g)$ is a Transformer over a summary token, the context tokens, the embedded code tokens, and the patch tokens of a goal image g . It outputs a scalar score from the summary token. It never receives the action, so the code is the only channel through which the action’s consequences reach it. The reader is trained on the K candidates branched from one state, whose futures and labels are known during training. It uses a pairwise logistic ranking loss over the pairs whose labels differ by more than a margin ϵ :

$$\mathcal{L}_{\text{rank}} = \mathbb{E}_{(i,j): \ell_i > \ell_j + \epsilon} [\log(1 + e^{-(s_i - s_j)})], \quad (2)$$

$$s_i = D_\psi(C, S_i, g), \quad \ell_i = \ell_g(\tau_i).$$

Ranking within a bank matches how the score is used: it compares candidates from one state and need not be calibrated across states. These labels are the only task supervision in CTA, and they are used in training only.

Feature decoder. A ranking loss alone lets the code shrink to whatever orders the training goals, which would not transfer to new goals. The feature decoder $G_\psi(C, S)$ therefore reconstructs the future tokens from the context and the code. One learned query per future token position cross-attends to the context and code tokens, and the decoder predicts how each token changes from the corresponding token of the current image. Its loss $\mathcal{L}_{\text{feat}}$ is the squared error divided by that of a copy baseline that predicts no change, computed separately for the final and the intermediate images, so a code that carries nothing about the change scores one. In JEPA terms, the feature decoder is the regularizer that keeps the target representation from collapsing [24]. The world model’s prediction target remains the code, and no image is ever generated.

4.4. Action-conditioned world model

The world model $p_\theta(S | C, \mathbf{A})$ is the predictor of the joint-embedding architecture. A Transformer encoder attends over the context tokens and one token per executed action. A Transformer decoder then generates the M code tokens autoregressively as categorical distributions over the vocabulary, which preserves the dependencies between code positions. The world model never receives future images, future proprioception, the goal, or any label. It is trained by teacher forcing against stop-gradient codes of the target encoder,

$$\mathcal{L}_{\text{WM}} = \mathbb{E}[-\log p_\theta(\text{sg}(S) | C, \mathbf{A})]. \quad (3)$$

A context-only model $r_\omega(S | C)$ with the same architecture but no action tokens is trained alongside. It estimates the rate of the code. Its excess cross-entropy over the world model

estimates $I(S; \mathbf{A} | C)$, the part of the code that the action explains; a code that ignores the action would show none. At test time we decode the most likely code greedily. We also evaluate two variants: the expected FSQ codeword under the world model at each position, given the greedy prefix, and the reader’s scores averaged over four sampled codes.

4.5. Training

Data. Training tuples come from rollouts of the frozen proposal policy. At every decision of a rollout, all K proposals are executed for H_e steps from clones of the simulator state. This yields K futures and their labels from an identical history, and the rollout then continues with the default proposal. All branches of a root episode belong to the same data split. Branching is what lets both the ranking loss and the world model observe different actions from the same history.

Stage 1: code. The target encoder, reader, and feature decoder are trained jointly on actual futures with

$$\mathcal{L}_{\text{code}} = \mathcal{L}_{\text{rank}} + \alpha \mathcal{L}_{\text{feat}}. \quad (4)$$

One goal image per decision is sampled from the training goals, and gradients pass through the quantizer with the straight-through estimator.

Stage 2: world model. With the target encoder frozen, the world model and the context-only model are trained on its codes by (3).

Stage 3: co-design (optional). A code that retains the future well may still be hard to forecast. In this stage, fixed blocks of code updates alternate with fixed blocks of world-model updates. Code updates add a predictability term under a frozen copy $\bar{p}_\theta$ of the world model:

$$\mathcal{L}_{\text{code}}^\lambda = \mathcal{L}_{\text{rank}} + \alpha \mathcal{L}_{\text{feat}} + \lambda \|S - \mathbb{E}_{\bar{p}_\theta(\cdot | C, \mathbf{A})}[S]\|^2. \quad (5)$$

The last term is a differentiable surrogate of $-\log \bar{p}_\theta(S | C, \mathbf{A})$. It pulls each quantized token, viewed as its FSQ codeword, toward the codeword the frozen world model expects given the preceding tokens. World-model updates use targets from an exponential moving average of the target encoder, which replaces the encoder at the end. Setting $\lambda=0$ reduces training to stages 1 and 2, and this ablation tests whether predictability-aware co-design adds anything. **[TODO: Optional reader adaptation (fine-tuning D_ψ on a mix of target and predicted codes with everything else frozen) is a debugging option; describe it here only if the locked configuration uses it.]**

Collapse diagnostics. A code can satisfy the losses while ignoring the action or the future. We monitor code perplexity, the fraction of distinct codes within a bank, the feature decoder’s R^2 relative to the copy baseline, and the action information $I(S; \mathbf{A} | C)$.

Algorithm 1 Policy-guided planning with CTA.

Require: frozen policy π ; frozen encoder φ ; world model p_θ ;
 reader D_ψ ; goal images $\mathcal{G}$; K ; H_e

- 1: **while** episode not finished **do**
- 2: $C \leftarrow$ features of h_t under φ $\triangleright$ once per decision
- 3: $\mathbf{A}^{(k)} \leftarrow \pi(h_t; \xi_k)$ for $k = 1, \dots, K$ $\triangleright$ shared bank
- 4: $\hat{S}^{(k)} \leftarrow$ greedy decode of $p_\theta(\cdot | C, \mathbf{A}^{(k)})$ $\triangleright$ batched
- 5: $s_k \leftarrow \frac{1}{|\mathcal{G}|} \sum_{g \in \mathcal{G}} D_\psi(C, \hat{S}^{(k)}, g)$
- 6: $k^* \leftarrow$ smallest index attaining $\max_k s_k$ $\triangleright$ ties keep the default
- 7: execute $\mathbf{A}_{1:H_e}^{(k^*)}$; $t \leftarrow t + H_e$

4.6. Planning with predicted codes

Algorithm 1 summarizes deployment. The planning score of a candidate is the reader’s score averaged over the goal images, which are final frames of successful episodes of the task. All candidates are scored from the context and their actions, before any of them is executed. Per decision, CTA computes the image features once, runs the world model on the K candidates in one batch, and runs the reader on $K|\mathcal{G}|$ candidate–goal pairs. Because codes do not depend on the goal, a new goal on the same candidates costs only additional reader evaluations.

4.7. Implementation details

All modules have width 256 and 8 attention heads. The target encoder has three decoder layers, the reader four encoder layers, and the feature decoder two decoder layers. The world model and the context-only model each have four encoder and four decoder layers. Unless stated otherwise, $M=16$ (128 bits per candidate), $\alpha=1$, and $\epsilon=10^{-3}$. We train with AdamW (learning rate 3×10^{-4} , weight decay 0.05, 500 warm-up steps, cosine decay) in bfloat16. Stage 1 runs 10k steps with 16 decisions (128 candidates) per batch, and stage 2 runs 20k steps with 32 decisions per batch. Stage 3 alternates 200 code steps with 400 world-model steps at learning rate 10^{-4} , with EMA rate 0.99. On PushT, images are 96×96 and resized to 224×224 for DINOv2, $H_e=8$, the intermediate images are those after steps 2, 4, and 6, and $K=8$. Each action is a target position of the agent, embedded both in absolute coordinates and relative to the current position. **[TODO: LIBERO-Goal: action embedding, H_e , intermediate frames, camera views.]**

5. Experiments

We ask four questions. (i) Does a released policy’s proposal bank contain better candidates, and can a generic feature distance find them (Sec. 5.2)? (ii) Does CTA select better than the policy and than matched baselines in closed loop (Sec. 5.3)? (iii) Where along the chain from actual future to predicted code is ranking lost (Sec. 5.4)? (iv) Which design

choices matter, and does a predicted code transfer to held-out goals better than a direct scorer (Secs. 5.5 and 5.6)?

5.1. Setup

Arenas and policies. *PushT* [14]: we use the released LeRobot Diffusion Policy [8], which observes two frames, denoises a 16-step action tensor, and executes 8 actions. An episode succeeds when the block covers more than 95% of the target within 300 steps. The unmodified policy succeeds on 65 of 100 evaluation episodes; the model card reports 65.4%. *LIBERO-Goal* [26]: ten goals in one kitchen scene, with the released SmolVLA [35] checkpoint executing 10 actions per chunk. With the unmodified evaluation script it reaches 71.3% over three seeds, below the 91% reported; we use the policy as measured and do not tune it. **[TODO: LIBERO branching harness, clone fidelity, and the split into training and held-out goals.]**

Proposal banks. At every decision the policy draws $K=8$ chunks from independently seeded noise; the bank is shared by all methods, and $\mathbf{A}^{(1)}$ is the policy’s own draw. Simulator clones used for branching are verified to reproduce the live continuation of the episode.

Data and splits. On PushT, training uses 800 policy roll-outs with all eight proposals branched at every decision. Separate episodes serve offline development (100), closed-loop development (200), and a sealed confirmation set (400). The confirmation set is opened once, for the locked configuration trained with three seeds. Goal images are the final frames of 16 successful episodes outside all splits.

Compared scorers. All scorers rank the same bank. *Policy only* executes $\mathbf{A}^{(1)}$. The *oracle* simulates every candidate and picks the highest 8-step progress label; it is privileged and bounds myopic selection. The *actual-future reader* is a reader of the same size, trained with the same data and loss on the uncompressed actual future; it is not deployable and bounds any predictor of the future. *Code of the actual future* applies CTA’s reader to the target encoder’s code of the simulated future. The *direct scorer* $D(C, \mathbf{A}, g)$ has eight layers and is trained with the same data, loss, and steps. The *per-step latent world model* is a DINO-WM-style predictor of the patch features of every future step, trained on the same data and read by the actual-future reader. We also compare with *GPC* [33]. **[TODO: Per-step latent world model and GPC are not implemented yet.]** Finally, *feature distance* scores the actual final frame by its negative mean squared DINOv2 patch-feature distance to the mean feature map of the goal images, which is DINO-WM’s planning cost under perfect prediction.

Metrics. The primary metric is closed-loop success. Differences are paired by episode, with bootstrap confidence intervals (10,000 resamples) and exact McNemar tests. Offline, on development decisions, we report the within-bank

Selection among $K=8$ proposals	Set A (100 ep.)	Set B (400 ep.)
Policy, unmodified inference	65	—
Policy only ($\mathbf{A}^{(1)}$, our sampler)	64	62.5
Bank medoid (no future information)	68	—
Feature distance on the true future	67	—
Learned reader on the true future	—	71.0
Oracle, 8-step progress (privileged)	79	76.5

Table 2. **PushT headroom**: closed-loop success (%) when selecting among the policy’s own proposals, on two disjoint sets of episodes. The two rows on the true future are not deployable; they bound what a world model that predicted the future perfectly could obtain with that scorer.

Spearman correlation between a score and the 8-step label. We also report the retained gap: the fraction of the oracle’s per-decision label gain over $\mathbf{A}^{(1)}$ that the scorer’s choice obtains.

5.2. Headroom in the proposal bank

Table 2 shows that the bank has headroom. On set B, selecting by the privileged oracle raises success by 14 points, 95% CI [8.8, 19.3] (94 episodes solved only by the oracle, 38 only by the policy; McNemar $p < 10^{-5}$). On set A the gain is 15 points [5, 25], and on a third set of 100 episodes it was 10 points [−1, 22]. Per decision the gain is small: a strictly better candidate exists at 54% of decisions, and the oracle’s choice improves coverage by only 0.007 on average. Swapping a single chunk barely changes the outcome (+2 points [−3, 7.5] in a split-seed estimate). Success therefore comes from being slightly better at many decisions, and a scorer must resolve small differences in progress at every one of them.

Feature distance fails to do so even on the true future. On set A it gains 3 points [−8, 14], and its within-bank Spearman correlation with the label is 0.06 over 1,742 decisions. A per-step latent world model that plans with this cost is thus capped near the policy even under perfect prediction. A reader trained with the ranking loss of Sec. 4.3 on the true future does much better. On set B it gains 8.5 points [2.8, 14.3] (McNemar $p = 0.005$), 61% of the oracle’s gain, with a within-bank Spearman correlation of 0.23. This learned reader on the true future is the upper bound that CTA approaches from compressed and then predicted futures.

5.3. Closed-loop results

[Main result: CTA vs. policy only (primary contrast), vs. the direct scorer, and retention relative to the actual-future reader and the oracle.]

	PushT	LIBERO-Goal seen held-out	ms per decision
Policy only	—	—	—
Oracle [†]	—	—	—
Actual-future reader [†]	—	—	—
Direct scorer	—	—	—
Per-step latent WM	—	—	—
GPC [33]	—	—	—
CTA (ours)	—	—	—

Table 3. **Closed-loop success** (%) on sealed episodes, averaged over three training seeds, and planning latency per decision, including proposal sampling. [†]Not deployable: simulates every candidate.

Scorer input	Spearman	Retained gap
Actual future (uncompressed)	—	—
Code of the actual future	—	—
Predicted code, greedy	—	—
Predicted code, expected codeword	—	—
Predicted code, 4 samples	—	—
Direct scorer (actions)	—	—

Table 4. **Attribution ladder** on PushT development decisions: within-bank Spearman correlation with the 8-step label and retained gap, with episode-clustered 95% intervals. A drop from one row to the next locates the loss: compression (row 1 to 2) or prediction (row 2 to 3).

5.4. Attribution ladder

Table 4 scores the same decisions with the same reader architecture as the input moves from the actual future, to its code, to the code predicted from actions. [Interpretation; code diagnostics: perplexity, distinct codes per bank, $I(S; \mathbf{A} \mid C)$ in bits, feature R^2 .]

5.5. Ablations

Each ablation changes one design choice, at the same history, proposal bank, data, and training budget.

- *Conditioning on history*. The target encoder does not see C , at the same number of bits. The heads still see C .
- *Segment versus endpoint*. The target encoder sees only the final image. A decision is made once per chunk, but some differences between chunks lie in the path.
- *Segment versus per-step codes*. A conditional code per time step with the same context and the same total number of bits. If it matches CTA, the segment claim fails. [TODO: Not implemented.]
- *Feature decoder*. $\alpha=0$ tests whether reconstruction is what keeps the code useful for goals outside training.
- *Predictability*. $\lambda=0$ versus co-design with $\lambda>0$ in (5).
- *Rate*. $M \in \{8, 16, 32\}$, which traces the accuracy–latency frontier.

[Ablation table.]

5.6. Held-out goals

The advantage of predicting a goal-independent code over scoring actions directly should appear on goals that were not seen in training. On LIBERO-Goal, the reader and the direct scorer are trained with labels for a subset of the ten goals, and both are evaluated on the remaining goals with the same world model and bank. [Held-out goal result.]

6. Conclusion and Limitations

[Conclusion, written after results.]

Limitations. The reader is trained with task-progress labels from the simulator, so CTA is not label-free, although the world model never sees a label and nothing uses labels at test time. The proposal policy is a pretrained imitation-learning model, so the complete system is not free of expert data. Training data require branching the simulator at every decision. PushT has a single fixed goal and cannot test transfer across goals; that claim rests on LIBERO-Goal. Our LIBERO-Goal policy reproduces below its reported success rate.

References

- [1] Dilip Arumugam and Benjamin Van Roy. Deciding what to model: Value-equivalent sampling for reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 3
- [2] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 1, 2
- [3] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. 1, 2
- [4] Johannes Ballé, Valero Laparra, and Eero P. Simoncelli. End-to-end optimized image compression. In *International Conference on Learning Representations (ICLR)*, 2017. 3
- [5] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *Transactions on Machine Learning Research (TMLR)*, 2024. 2
- [6] André Barreto, Will Dabney, Rémi Munos, Jonathan J. Hunt, Tom Schaul, Hado van Hasselt, and David Silver. Successor features for transfer in reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. 2
- [7] Boyuan Chen, Chuning Zhu, Pulkit Agrawal, Kaiqing Zhang, and Abhishek Gupta. Self-supervised reinforcement learning that transfers using random features. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. 2
- [8] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023. 2, 6
- [9] Hanchen Cui et al. DreamSteer: Latent world models can steer VLA policies during deployment without any finetuning. *arXiv preprint arXiv:2607.02865*, 2026. 3
- [10] Zihan Ding, Amy Zhang, Yuandong Tian, and Qingqing Zheng. Diffusion world model: Future modeling beyond step-by-step rollout for offline reinforcement learning. *arXiv preprint arXiv:2402.03570*, 2024. 2
- [11] Maximilian Du and Shuran Song. DynaGuide: Steering diffusion policies with active dynamic guidance. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. 3
- [12] Jesse Farebrother, Matteo Pirodda, Andrea Tirinzoni, Rémi Munos, Alessandro Lazaric, and Ahmed Touati. Temporal difference flows. In *International Conference on Machine Learning (ICML)*, 2025. 2
- [13] Jesse Farebrother, Matteo Pirodda, Andrea Tirinzoni, Marc G. Bellemare, Alessandro Lazaric, and Ahmed Touati. Compositional planning with jumpy world models. *arXiv preprint arXiv:2602.19634*, 2026. 2
- [14] Pete Florence, Corey Lynch, Andy Zeng, Oscar Ramirez, Ayzaan Wahid, Laura Downs, Adrian Wong, Johnny Lee, Igor Mordatch, and Jonathan Tompson. Implicit behavioral cloning. In *Conference on Robot Learning (CoRL)*, 2021. 2, 6
- [15] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning (ICML)*, 2023. 3
- [16] Christopher Grimm, André Barreto, Satinder Singh, and David Silver. The value equivalence principle for model-based reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. 3
- [17] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning (ICML)*, 2019. 1, 2
- [18] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 2025.
- [19] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations (ICLR)*, 2024. 2
- [20] Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. 3
- [21] Dongwon Kim, Gawon Seo, Jinsung Lee, Minsu Cho, and Suha Kwak. Planning in 8 tokens: A compact discrete tokenizer for latent world model. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026. 2
- [22] Jacky Kwok, Christopher Agia, Rohan Sinha, Matt Foutter, Shulu Li, Ion Stoica, Azalia Mirhoseini, and Marco Pavone. RoboMonkey: Scaling test-time sampling and verification

- for vision-language-action models. In *Conference on Robot Learning (CoRL)*, 2025. 3
- [23] Nathan O. Lambert, Albert Wilcox, Howard Zhang, Kristofer S. J. Pister, and Roberto Calandra. Learning accurate long-term dynamics for model-based reinforcement learning. In *IEEE Conference on Decision and Control (CDC)*, 2021. 2
- [24] Yann LeCun. A path towards autonomous machine intelligence. OpenReview preprint, version 0.9.2, 2022. 1, 2, 5
- [25] Jiahao Li, Bin Li, and Yan Lu. Deep contextual video compression. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. 2, 3
- [26] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023. 2, 6
- [27] Fabian Mentzer, David Minnen, Eirikur Agustsson, and Michael Tschannen. Finite scalar quantization: VQ-VAE made simple. In *International Conference on Learning Representations (ICLR)*, 2024. 4
- [28] Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations (ICLR)*, 2023. 1, 2
- [29] Vincent Micheli, Eloi Alonso, and François Fleuret. Efficient world models with context-aware tokenization. In *International Conference on Machine Learning (ICML)*, 2024. 2
- [30] Mitsuhiro Nakamoto, Oier Mees, Aviral Kumar, and Sergey Levine. Steering your generalists: Improving robotic foundation models via value guidance. In *Conference on Robot Learning (CoRL)*, 2024. 1, 3
- [31] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research (TMLR)*, 2024. 2, 4
- [32] Xiangyuan Peng, Xingsi Dong, and Si Wu. Vector quantization in the brain: Grid-like codes in world models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. 2
- [33] Han Qi, Haocheng Yin, Aris Zhu, Yilun Du, and Heng Yang. Inference-time enhancement of generative robot policies via predictive world modeling. *IEEE Robotics and Automation Letters (RA-L)*, 2025. 3, 6, 7
- [34] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020. 3
- [35] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, et al. SmoVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025. 6
- [36] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999. 2
- [37] Yilin Wu, Ran Tian, Gokul Swamy, and Andrea Bajcsy. From foresight to forethought: VLM-in-the-loop policy steering via latent alignment. In *Robotics: Science and Systems (RSS)*, 2025. 3
- [38] Aaron D. Wyner and Jacob Ziv. The rate-distortion function for source coding with side information at the decoder. *IEEE Transactions on Information Theory*, 22(1):1–10, 1976. 3
- [39] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *International Conference on Machine Learning (ICML)*, 2025. 1, 2, 4